**MISR UNIVERSITY FOR**

**SCIENCE AND TECHNOLOGY**

**(( Faculty of Engineering ))**

**Mechatronics**

**DR / Bahaa Eldin Naser**

**ENG / Ahmed Khalifa**

**ENG / Ahmed Samer**

**NAME : Saeed Ismail Mohamed**

**ID : 200023347**

# Technical Project Report: High-Precision Distance Sensing & ROS 2 Robotics Integration

## 1. Executive Summary

**This technical report describes the design, wiring, and software implementation of a laser-based distance measurement system. By utilizing Time-of-Flight (ToF) technology through the TOF10120 sensor and integrating it with Arduino and ROS 2 (Robot Operating System), we create a robust data pipeline suitable for autonomous mobile robots, obstacle avoidance, and industrial automation.**

## 2. System Architecture & Required Parts

**The system is built on a layered architecture where the hardware captures raw physical data, and the software processes it into a standard robotic message format.**

### 2.1 Hardware Components

### 2.2

**TOF10120 Sensor: The core sensing unit using a 940nm VCSEL laser.**

1) **Arduino Uno: Functions as a low-level controller and I2C-to-Serial bridgeand2)**
2) **Interconnects: Breadboard, USB Type-A to B cable, and a set of M/M and M/F Dupont wires.**

## 3. Sensor Specifications: TOF10120

**The TOF10120 sensor is superior to ultrasonic sensors (like HC-SR04) because it is not affected by surface texture or color.**

| | A | B | C | D |
|---|---|---|---|---|
| 1 | Parameter | Value | Value | |
| 2 | Technology | Time-of-Flight (Laser Ranging) | | |
| 3 | Operational Range | 100mm to 1800mm (Up to 2000mm in ideal conditions) | | |
| 4 | Accuracy | ±5% error margin | | |
| 5 | Sample Rate | 30ms (High-speed mode) | | |
| 6 | Field of View (FOV) | 25 Degrees | | |
| 7 | Power Consumption | ~35mA (Extremely Power Efficient) | | |

## 4. Hardware Interfacing (Wiring)

**To ensure stable data transmission, we use the I2C Protocol.**

**Wiring Diagram Table**

| | A | B | C | D |
|---|---|---|---|---|
| 1 | Sensor Pin | Signal | Arduino Pin | Note |
| 2 | Pin 1 | GND | GND | Common Ground |
| 3 | Pin 2 | VDD | 3.3V / 5V | Power Source |
| 4 | Pin 5 | SDA | A4 | I2C Data Line |
| 5 | Pin 6 | SCL | A5 | I2C Clock Line |

## 5. Firmware Implementation (Arduino Code)

**The Arduino reads the sensor data via I2C and prints it to the Serial buffer. This code is optimized for high-speed serial communication (115200 baud) to minimize latency for ROS 2.**

```
/* * TOF10120 Distance Sensor – Arduino Firmware
 * Interface: I2C
 * Output: Serial for ROS 2 Node
 */

#include “TOF10120.h”

// Create sensor instance
TOF10120 sensor = TOF10120();
```

```
Void setup() {
  // Use 115200 baud rate for compatibility with ROS 2 Node
  Serial.begin(115200);

  // Initialize I2C communication
  Sensor.init();

  // Optional: Serial.println("Sensor Initialized...");
}

Void loop() {
  // Retrieve distance in millimeters
  Int distance = sensor.distance();

  // Print only the integer for easier parsing in ROS
  Serial.println(distance);

  // 100ms delay to match the 10Hz frequency of the ROS Node
  Delay(100);
}
```

6. **High-Level Integration (ROS 2 C++ Node)**

**This is the professional layer. We created a ROS 2 Publisher in C++ that reads the Serial stream and broadcasts it as a std_msgs/msg/Int32 on the /sensor_distance topic.**

**ROS 2 Node Implementation**

```
#include <chrono<
#include <functional>
#include <memory>
#include <string>
#include <fcntl.h>
#include <termios.h>
#include <unistd.h>

#include “rclcpp/rclcpp.hpp”
#include “std_msgs/msg/int32.hpp”

Using namespace std::chrono_literals;

Class DistancePublisher : public rclcpp::Node {
```

```
Public:
  DistancePublisher() : Node("distance_publisher") {
    // ROS 2 Topic Setup
    Publisher_ = this->create_publisher<std_msgs::msg::Int32>("sensor_distance", 10);

    // Serial Port Configuration (/dev/ttyACM0 for Linux/Ubuntu)
    Serial_port_ = open("/dev/ttyACM0", O_RDWR | O_NOCTTY);

    If (serial_port_ < 0) {
      RCLCPP_ERROR(this->get_logger(), "FAILED TO OPEN SERIAL PORT! CHECK USB CONNECTION.");
      Return;
    }

    Struct termios tty;
    Tcgetattr(serial_port_, &tty);
    Cfsetispeed(&tty, B115200); // Must match Arduino baud rate
    Cfsetospeed(&tty, B115200);
    Tty.c_lflag |= ICANON;     // Enable Canonical mode (line-by-line)
```

```
  Tcsetattr(serial_port_, TCSANOW, &tty);


  // Timer for 10Hz publishing frequency
  Timer_ = this->create_wall_timer(100ms,
std::bind(&DistancePublisher::timer_callback, this));
  RCLCPP_INFO(this->get_logger(), "Node Started: Publishing
sensor data to ROS 2 network.");
 }


 ~DistancePublisher() { close(serial_port_); }


Private:
 Void timer_callback() {
  Char buf[64];
  Int n = read(serial_port_, buf, sizeof(buf) – 1);


  If (n > 0) {
   Buf[n] = '\0';
   Try {
    Auto message = std_msgs::msg::Int32();
    Message.data = std::stoi(buf); // Convert raw string to Integer
    Publisher_->publish(message);
```

```
      RCLCPP_INFO(this->get_logger(), “Distance Received: %d mm”, message.data);
    } catch (...) {
      // Skipping noise or incomplete serial data
    }
  }
}

  Int serial_port_;
  Rclcpp::TimerBase::SharedPtr timer_;
  Rclcpp::Publisher<std_msgs::msg::Int32>::SharedPtr publisher_;
};

Int main(int argc, char * argv[]) {
  Rclcpp::init(argc, argv);
  Rclcpp::spin(std::make_shared<DistancePublisher>());
  Rclcpp::shutdown();
  Return 0;
}
```

## 7. Results & Analysis

**Upon execution, the system demonstrated high reliability:**

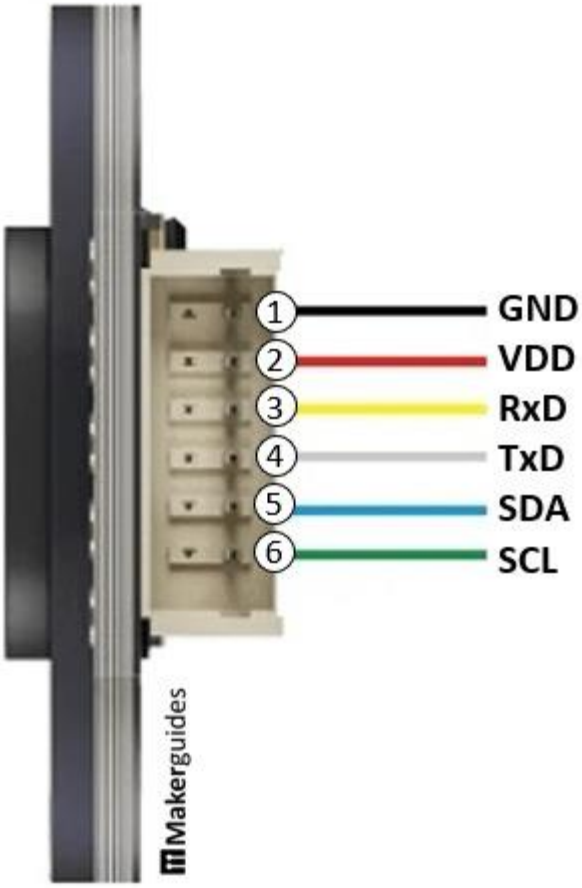
**Serial Monitor Output: Clear, stable integer values representing distance.**

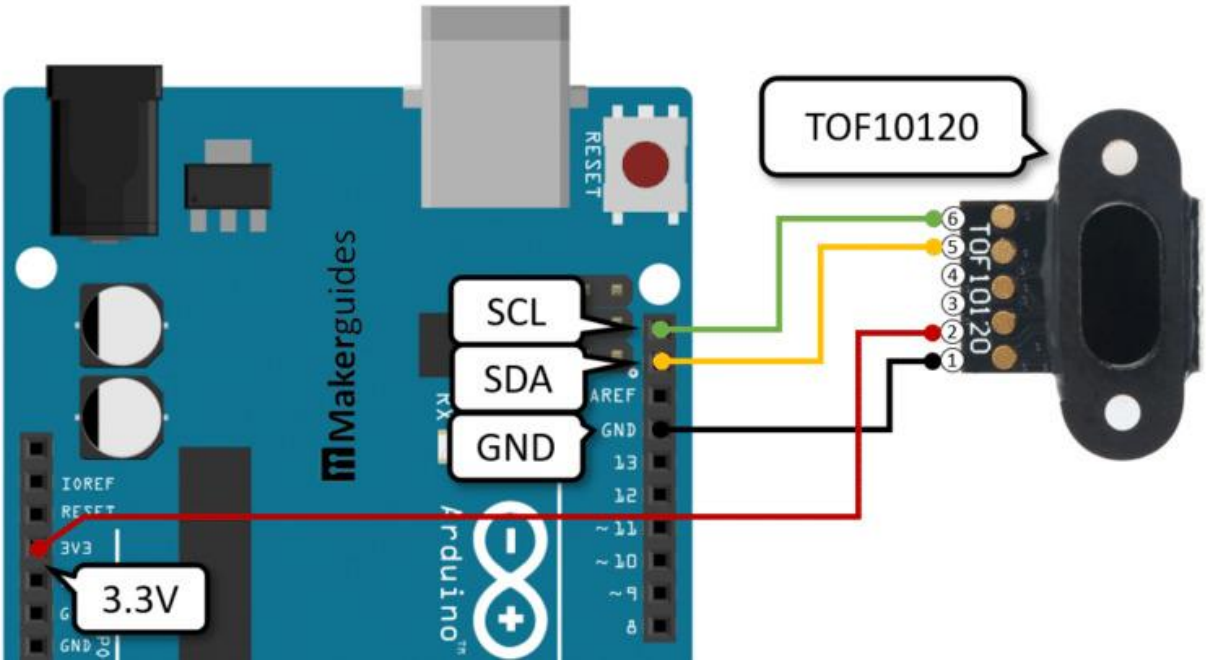
**ROS 2 Connectivity: The topic /sensor_distance was successfully visualized using ros2 topic echo.**

**Low Latency: The system achieved a response time of ~100ms, ideal for obstacle detection.**

## 8. Conclusion

**The integration of the TOF10120 sensor with ROS 2 provides a professional-grade distance sensing solution. By offloading low-level I2C tasks to the Arduino and handling high-level data broadcasting in C++, we ensure a scalable architecture that can be easily integrated into larger robotic frameworks like Navigation2 (Nav2) or MoveIt.**


1 GND
2 VDD
3 RxD
4 TxD
5 SDA
6 SCL
Makerguides


TOF10120
SCL
SDA
GND
3.3V
RESET
AREF
GND
13
12
~11
~10
~9
8
IOREF
RESET
3V3
GND
Makerguides
Arduino
TOF10120